Python eBPF: BCC C-String Runtime vs Python-BPF AST AOT

Nizar Akbar Meilani — DomaiNesia

PyCon Taiwan 2026

About Me

Systems engineer at DomaiNesia · Yogyakarta, Indonesia

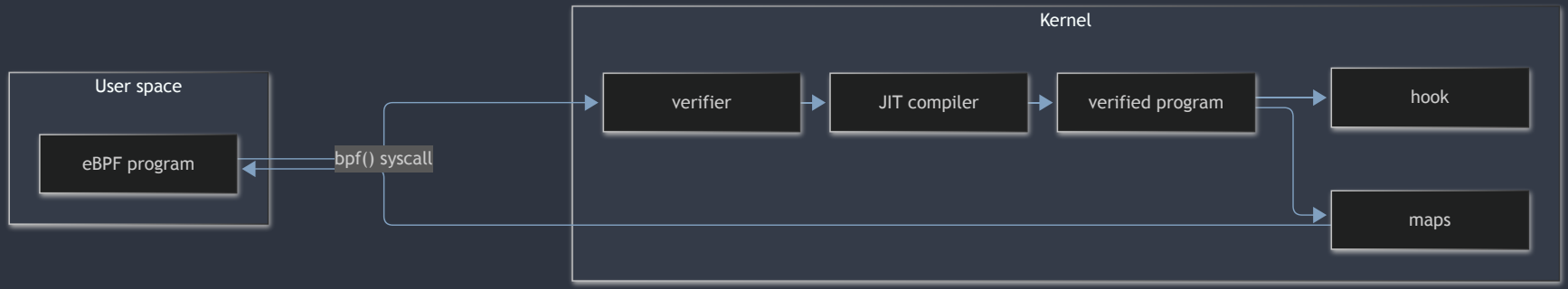
- Focus: eBPF from Python — kernel observability without writing C
- Recently: Python wrappers for traffic filtering — utilizing bpfilter
- Today: PyCon Taiwan 2026 · previously PyCon APAC 2024 · PyCon HK 2025 · PyCon ID 2025



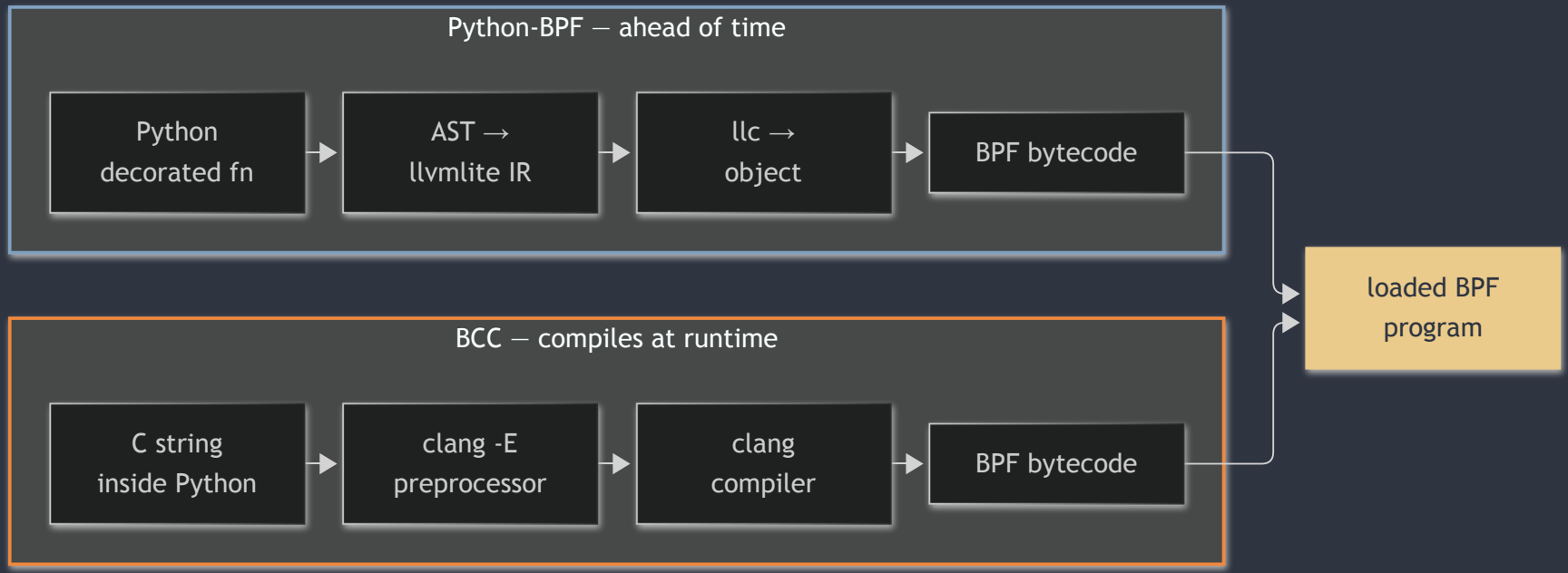
https://github.com/nizarakbar

https://github.com/nizarakbarm/pycon-tw-2026-bcc-vs-pythonbpf

eBPF architecture overview



Architecture: How a Program Becomes Bytecode



Consequence: BCC needs `clang` + kernel headers on the host at runtime. Python-BPF needs neither once built.

Two frameworks, three questions

What do maps do? → state, and a channel to userspace or another program

How does streaming data reach userspace? → perf event array

How do you emit an event? → submit

State: the HashMap

BCC — C macro, compiled at runtime

```c
BPF_HASH(start, u32);   // key=u32, leaf=u64, size=10240

int probe(struct pt_regs *ctx) {
    u32 key = bpf_get_current_pid_tgid();
    u64 *val = start.lookup(&key);   // ptr to leaf
    if (val) start.update(&key, val);
    return 0;
}
```

Python-BPF — Python class, AOT

```python
@bpf
@map
def start() -> HashMap:
    return HashMap(key=c_uint32, value=c_uint64,
                   max_entries=10240)

@bpf
@section("tracepoint/...")
def probe(ctx):
    key = pid()
    val = start.lookup(key)          # value, not ptr
    if val: start.update(key, val)
    return 0
```

Streaming to userspace: the perf event array

BCC — `BPF_PERF_OUTPUT` + `perf_submit`

```
BPF_PERF_OUTPUT(events);

struct data_t { u32 pid; u64 ts; char comm[16]; };

int hello(struct pt_regs *ctx) {
    struct data_t d = {};
    d.pid = bpf_get_current_pid_tgid();
    d.ts  = bpf_ktime_get_ns();
    events.perf_submit(ctx, &d, sizeof(d));
    return 0;
}
```

Python-BPF — `PerfEventArray.output`

```
@bpf
@map
def events() -> PerfEventArray:
    return PerfEventArray(key_size=c_uint32,
                          value_size=c_uint32)

@bpf
@section("tracepoint/...")
def hello(ctx):
    d = ProcessEvent()
    d.pid = pid()
    d.ts  = ktime()
    events.output(d)     # sizeof() inferred
    return 0
```

Same job, different spelling

step	BCC	Python-BPF
declare map	`BPF_HASH(start, u32)`	`HashMap(key=c_uint32, …)`
lookup	`map.lookup(&key)` → `u64*`	`map.lookup(key)` → value
update	`map.update(&key, &val)`	`map.update(key, val)`
emit	`events.perf_submit(ctx,&d,sizeof(d))`	`events.output(d)`
define event	`struct data_t {…}`	`@struct class Event: …`
userspace poll	`b.open_perf_buffer(cb)` + `perf_buffer_poll()`	same call, `open_perf_buffer(cb)` + `perf.poll()`

Which program starts faster?

	BCC	Python-BPF
init CPU	0.4 s	0.02 s
child process	none	llc
header blocks	20,768	160

BCC compiles C at runtime. Python-BPF does not.

What do we run?

item	value
tracepoint	`sys_enter_execve`
trigger	2M `fork` + `execve` pairs
program	`map_1_perf_timed_count.py`
host	vmnono (3 CPUs)

Both frameworks run the same program.

What does the callback cost?

- count — prints one line
- empty — does nothing
- print — prints a constant

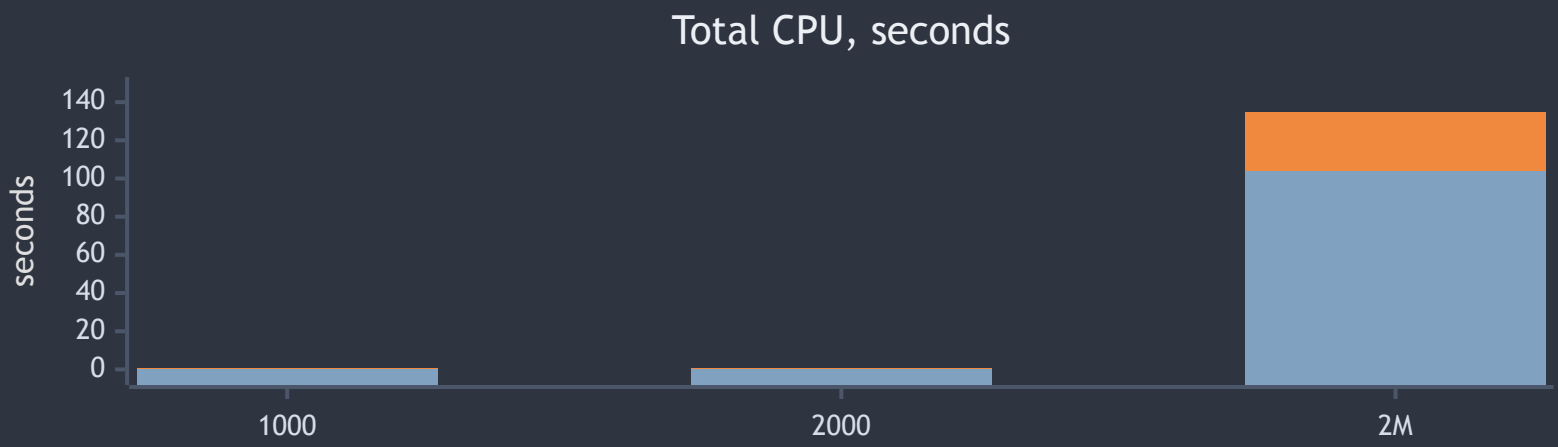
Same program. Only the callback changes.

Both frameworks deliver every event

	events delivered	trigger sent
BCC	2,000,000	2,000,000
Python-BPF	2,000,000	2,000,000

Both frameworks delivered 2,000,000 events. Nothing was lost.

Python-BPF uses less CPU



N	BCC	Python-BPF	ratio
1000	0.625 s	0.120 s	5.21
2000	0.716 s	0.194 s	3.69
2M	134.568 s	103.658 s	1.30

An empty callback still costs more

callback body	BCC CPU	Python-BPF CPU	ratio
`pass`	103.3 s	84.0 s	1.23

The callback does nothing. BCC still spends 19.2 s more.

Where the CPU time goes

frame family, on the stack	BCC	Python-BPF
ctypes + libffi	84.0%	0%
clang	0.4%	0%
pybind11	0%	92.6%

`perf record`, 2M count arm. Share of sampled cycles with that frame anywhere on the stack.

Python-BPF binds with **pybind11** · BCC binds with **ctypes**

libffi is import cost, not eBPF cost

`readelf -d /usr/lib64/libbcc.so.0.36.0 | grep NEEDED` — no ffi anywhere:

```
 libclang-cpp.so.21.1   libLLVM.so.21.1   libelf.so.1    libbpf.so.1
libstdc++.so.6         libm.so.6         libgcc_s.so.1  libc.so.6
```

libffi enters on a separate branch of the same process:

```
python3
+- bcc/__init__.py:17   import ctypes as ct
|   +- _ctypes ... so   →  libffi.so.8        IMPORT cost
+- bcc/libbcc.py        ctypes.CDLL("libbcc.so.0")
    +- libbcc.so.0     →  libbpf.so.1        eBPF cost
```

Two branches off CPython, no edge between them.

BCC calls gettid once per event

syscall	BCC	Python-BPF
gettid	1,907,735	3
perf buffer wait	`poll` 1,978,114	`epoll_wait` 2,037,562
bpf + perf_event_open	19	31

Same perf fds. Same 1 ms timeout. Same once per event.

The wait is not eBPF work. Next slide traces the descriptors.

`strace -f -T`, 2M count arm. Counts only, not durations.

Both wait on the same perf fds

BCC — own loader, plain `poll`

```
 perf_event_open  4  1.91 ms  fd 9,10,11
epoll_create1    1  0.03 ms  unused
poll  1,978,114  1292.6 s
  [{fd=9},{fd=10},{fd=11}], 3, 1
   = 0  101,370  timeout
   = 1 1,846,789
   = 2    28,922
   = 3     1,032
   = ?         1  EINTR
  0.622 ms with data / 1.375 timeout
```

Python-BPF — libbpf, epoll set

```
 perf_event_open  4  0.12 ms  fd 10,11,12
epoll_create1    2  0.02 ms  fd 3, 9
epoll_ctl        3  0.01 ms  fd 10,11,12 → 9
epoll_wait 2,037,562  1451.8 s
  (9, [], 3, 1)
   = 0  109,831  timeout
   = 1 1,919,724
   = 2     7,868
   = 3       139
  0.675 ms with data / 1.373 timeout
```

Return codes add up to the same story: `0` is the timer, `1 .. 3` are the perf fds. BCC's poll sees more fds ready per call.

Wall time shows no difference

	BCC	Python-BPF	difference
wall	1763.6 s	1768.6 s	5 s, 0.3%
CPU	134.6 s	103.7 s	30.9 s, 30%

The trigger sets the pace, not the framework.

What makes up wall time?

$$\text{wall} = pre + 0.5 + \underbrace{t_{init} + \boxed{t_{poll} \supset t_{trigger}}}_{\text{sequential, one process}}$$

term	value	order
`pre`	2 s / 1 s	bash, before launch
`0.5`	0.5 s	bash, retry gate
`t_init`	0.45 / 0.02 s	program, then
`t_poll`	1763.6 / 1768.6 s	contains `t_trigger`
`t_trigger`	~1400 s	inside `t_poll`

Only `t_init` and `t_poll` differ. The trigger dominates.

Limitations

Four gaps in Python-BPF, and one open question.

Limitation 1 — BCC enforces in-kernel, Python-BPF cannot

Same job, two enforcement paths:

	Python-BPF	BCC
raise the signal	userspace, after `trace_pipe`	`bpf_send_signal`, in kernel
helpers available	18	36
`bpf_send_signal`	absent	present
result	`SIGKILL` from userspace	`SIGKILL`, trigger `rc = -9`

Both kill the trigger. BCC needs no userspace hop.

Limitation 2 — Two sections, one register struct

`struct_pt_regs` contains union members. Two sections that share it break the debug metadata.

sections with `struct_pt_regs`	result
one	compiles
two	debug metadata recurses

`RecursionError` in `function_debug_info.py:73`.

Workaround: one section per file. `kprobe` and `kretprobe` cannot share a file.

Limitation 3 — Python-BPF cannot read a kernel string

`ctx.si` holds a `struct filename *`. Three attempts to read its name:

attempt	helper	result
read `struct filename.name`	`probe_read`	`Struct type … not found`
read into a local `str(256)`	`probe_read_str`	`Variable 'buf' not found`
read into `c_uint64`	`probe_read_str`	8 bytes — truncates

What works reads the pointer, not the string: `probe_read(name_str, 8, fname_ptr)` → `0×ffff8f2d4705c000`, a valid kernel address.

Not garbage — the second dereference is missing. `probe_read` gives an `i64`; `probe_read_str` wants an `i8*`. No `inttoptr` between them.

Limitation 4 — Two gaps in the translator

construct	what happens
`ctx.args[0]`	falls through, no code emitted
`count += 1`	a plain add, not atomic

A map counter can lose updates.

Any other limitation?

What else have you hit with Python-BPF?

Is this Python-BPF, or the kernel?

program shape	trace output
one `print`, three values	works
four or more values in one call	silent

Both frameworks. It is a kernel rule.

`bpf_trace_printk` takes three values after the format string.

The count is not the finding. The silence is.

Python-BPF loads, attaches, drops the line, and says nothing.

Do these limitations rule Python-BPF out anywhere?

Yes — for enforcement.

job	Python-BPF	why
measure, trace, count	yes	the event path is what we measured
deny a syscall	no	no deny helper, and BCC has none either
block before the call	no	a kill acts after it

And there are more we have not found. What have you hit?

For measurement, Python-BPF is the faster tool. For a deny job, choose one that can deny.

Where each framework wins

axis	BCC	Python-BPF	why
startup CPU	0.45 s	0.02 s	BCC compiles C at runtime
per-event CPU	134.6 s	103.7 s	ctypes and clang cost cycles
syscalls per event	1.96	1.03	one gettid per event in BCC
peak RSS	175 MB	22 MB	clang loaded in the BCC process

What to choose

pick Python-BPF	pick BCC
startup: 0.02 s against 0.45 s	a struct as a map value works today
CPU at 2M: 1.30x lower	
syscalls: 1.03 against 1.96 per event	
memory: 22 MB against 175 MB	

Boundary: Python-BPF cannot deny a syscall in the kernel.

Do you have any other idea based on this? Let us discuss.

Q&A

Slides, dataset and scripts: github.com/nizarakbarm/pycon-tw-2026-bcc-vs-pythonbpf

Python-BPF: github.com/python-bpf/python-bpf · BCC: github.com/iovisor/bcc